UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Faculty of Engineering, Built Environment and Information Technology

EAI 320

Intelligent Systems

Practical 3 Neural Network

Name and Surname	Student Number	Signature	% Contribution
J. Venter	19002409		50
A. van Niekerk	19007583		50

By signing this assignment we confirm that we have read and are aware of the University of Pretoria's policy on academic dishonesty and plagiarism and we declare that the work submitted in this assignment is our own as delimited by the mentioned policies. We explicitly declare that no parts of this assignment have been copied from current or previous students' work or any other sources (including the internet), whether copyrighted or not. We understand that we will be subjected to disciplinary actions should it be found that the work we submit here does not comply with the said policies.

May 28, 2021

Contents

1 Introduction 1
2 Background Theory 2
3 Task 1 5
3.1 Methodology 5
4 Task 2 6
4.1 Methodology 6
4.2 Results 8
4.3 Discussion 13
5 Task 3 15
5.1 Methodology 15
5.2 Results 15
5.3 Discussion 16
6 Conclusion 18

List of Figures

2.0.1 The structure of a neural network. 2
2.0.2 The flow of a neural network. 3
4.1.1 $\frac{1}{e^{-x}+1}$ 7
4.1.2 tanh(x) 7
4.1.3 ReLU(x) 7
4.2.1 a figure of batch size with the batch's accuracy and time 8
4.2.2 a figure of batch size with the batch's accuracy and time 9
4.2.3 a figure of learning rates with the neural network's accuracy and time 9
4.2.4 a figure of epochs and the corresponding error at a learning rate of 0 10
4.2.5 a figure of epochs and the corresponding error at a learning rate of 0.0005 10
4.2.6 a figure of epochs and the corresponding error at a learning rate of 0.05 10
4.2.7 a figure of epochs and the corresponding error at a learning rate of 1 11
4.2.8 a figure of error as a percentage for the first 1000 iterations of the data1 file . . 11
4.2.9 a figure of error as a percentage for the first 100000 iterations of the sorted data1 file at a learning rate of 0.05 12
4.2.10 a figure of time and accuracy over layer sizes with 4 neurons each 12
4.2.11 a figure of time and accuracy over neuron sizes with in 4 layers 12

1 Introduction

For this practical, the task is to implement a Neural Network (NN). The neural network then has to learn from a set of data that is provided in a csv file, from this set it will recognize which move is best to play after a series of moves have been played. After learning all of these possible outcomes it will then be tasked to compete against other bots in a Rock Paper Scissor game. The goal is to implement a neural network, optimize the neural network and report on what changes improve and do not improve the performance of the constructed neural network.

2 Background Theory

What is a neural network

a Neural network is an algorithm that mimics the behaviour of a human brain. A neural network is capable of recognizing patterns and solve common problems in the field of Artificial Intelligence.[1]

The structure of a neural network is inspired by the human brain, a neural network exists out of neurons that can be located in layers inside the neural network. Each neuron in the current layer is connected with weights to every neuron in the previous layer. The neural network improves by adjusting each weight of every neuron. If a neuron is on the correct path, its weights are rewarded (increased), if a neuron is on the incorrect path, its weights are not rewarded(decreased) the structure of a neural network can be seen in figure 2.0.1.2.0.1.

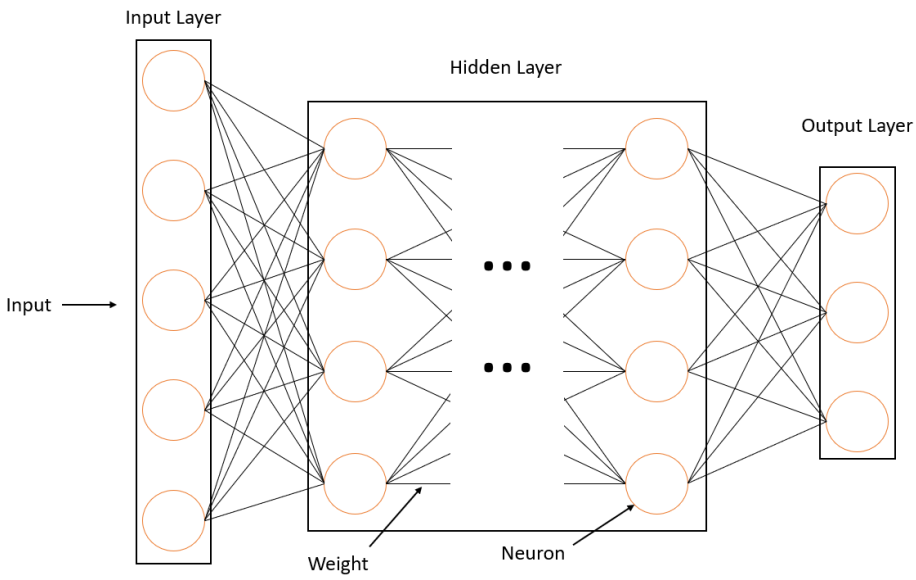

Figure 2.0.1: The structure of a neural network.

How does a neural network work

A neural network consists out of layers as described above, these layers can be divided into 3 main layers named the input layer, the hidden layer/s and the output layer each of these layers will be described in their separate section. A neural network improves because of a function called back-propagation which compares the output generated by the NN against the correct output and adjusts the weights of the neurons accordingly a figurative example of how a neural network "ticks" can be seen in figure 2.0.2 below.

What does a neuron do

a Neuron has a set number of inputs where the number of inputs depends on the number of neurons in the previous layer, each input has a weight associated with it. a Neuron has an activation function that either activates a neuron or does not activate a neuron, this activation function can be any function, however, it has been proven that non-linear activation functions tend to perform better in neural networks, for this practical, the activation function is

$$neuronoutput = tanh(i)$$

where I refer to the sum of all the inputs multiplied with their weights.

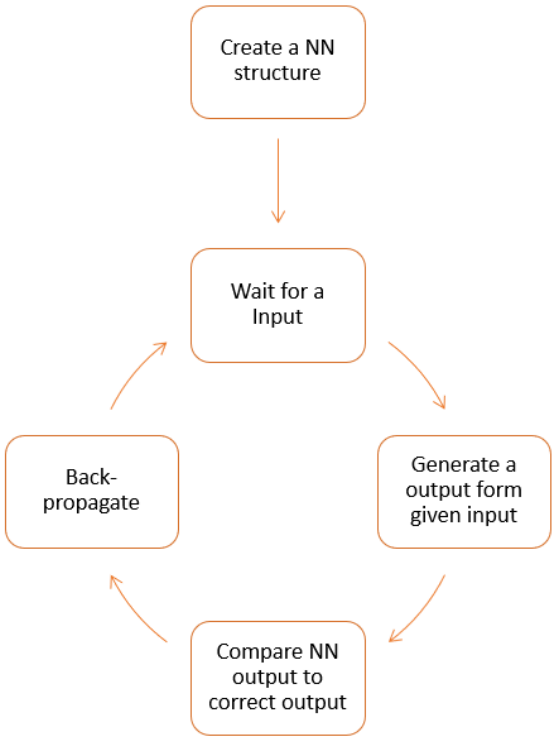

Figure 2.0.2: The flow of a neural network.

Creating a Neural Network structure

The structure of a neural network can seem to be very complexly connected but is quite simple, everything in the previous layer is connected to everything in the current layer. To create a NN structure one needs to construct several neurons, each neuron with several weights, where the number of weights corresponds to either the number of inputs for the input layer or the number of neurons in the previous layer, for the initialization process each weight for each neuron is initialized as a random value between 1 and -1. Add these neurons to the current layer and build the next layer from the known characteristics of this current layer. Therefore when one wants to construct the entire neural network one needs to create the input layer where the weights correspond to the number of inputs, then the hidden layers which can be of any size and finally the output layer which has 3 neurons in this case, each neuron representing a possible move.

Input to neural network output

The input layer receives the input from the user or program or the input as a history. these inputs cause certain neurons to fire(activate) in the input layer and from the input layer outputs it is passed into the hidden layer and causes hidden layer neurons to fire and finally from the hidden layer output it is passed into the output layer and causes certain neurons to fire, in the case of the Rock Paper Scissor structure, the neurons represent Rock or Paper or Scissors, therefore the higher of the 3 neurons is considered the final output.[1,0,0] for Rock ,[0,1,1] for Paper and [0,0,1] for Scissors.

Back-Propagation

The function of the back-propagation is to adjust the weights of each neuron so that when the function has the same input it will have a smaller error. The error can be defined as the difference between the desired output and the actual output this difference is used to adjust the weights of the neuron. The neural network also has a learning rate, the learning rate can be defined as the rate at which the neural network learns, meaning the larger the learning rate the more drastically the weights change if there is an error. along with the error and learning rate, the outputs of the current node and the sum of the inputs as well as the current weight in the current node will be used to modify the current weight to "learn". The formula for the changing if each separate weight, weight k for the jth neuron, can be expressed as:

$$\omega_{j,k} = \omega_{j,k} + \alpha \times a_j \times Err_k \times g'(\Sigma_i^{all} \omega_{i,j} \times a_i) \tag{1.1}$$

in equation 1.1 a_j refers to the output of this neuron (output of the activation function). g' is referred to as the gradient of the function or the derivative of the activation function, the sum of part can be calculated as all of the inputs multiplied by their weights for this neuron (the value that is inputted into the activation function also known as in_k).

When using the back-propagation function for the hidden layers and the input layers the function changes a little bit, the error and the g' function remains the same the just have to be used for the hidden layers as well therefore :

$$\Delta_k = Err_k \times g'(in_k) \tag{1.2}$$

by being able keep track of delta k this will be used in the function to change the hidden layer weights:

$$\Delta_j = g'(in_j) \times \Sigma_k^{all} \omega_{j,k} \times \Delta_k \tag{1.3}$$

by using the equation 1.3 we obtain the new weight j for neuron i as the following:

$$\omega_{i,j} = \omega_{i,j} + \alpha \times a_i \times \Delta_j \tag{1.4}$$

By implementing these formulas above for each neuron in the output layer and then propagating back to each neuron in each previous layer one changes every weight to a greater value if the weight has an influence that decreases the error and to a lower value if the weight has an influence that increases the error. by using the equations 1.1 and 1.4 one can fully implement the back-propagation for each layer.

3 Task 1

3.1 Methodology

The backpropagation function of a neural network is a function that improves the neural network. The backpropagation function improves the neural network by increasing or decreasing the weight of a neuron I to the next neuron j. The implementation of backpropagation is simple, using the math in section 2, it can calculate all of these values, the only difficult part is keeping track of all the variables and implementing it for a batch structure as well, the math stays the same the only difference is, matrices have to be used to keep track of all the inputs over all the neurons over all the layers. for a batch size of b the new formulas will be calculated as follows:

$$\begin{bmatrix} \Delta_{k1} \\ \Delta_{k2} \\ \Delta_{k3} \\ . \\ . \\ . \\ \Delta_{kb} \end{bmatrix} = \begin{bmatrix} Err_{k1} \times g'(in_{k1}) \\ Err_{k2} \times g'(in_{k2}) \\ Err_{k3} \times g'(in_{k3}) \\ . \\ . \\ . \\ Err_{kb} \times g'(in_{kb}) \end{bmatrix} \tag{1.5}$$

By using this matrix we can calculate $\omega_{j,k}$ as the following:

$$\begin{bmatrix} \omega 1_{j,k} \\ \omega 2_{j,k} \\ \omega 3_{j,k} \\ . \\ . \\ . \\ \omega b_{j,k} \end{bmatrix} = \begin{bmatrix} \omega_{j,k} + \alpha \times a_{j1} \times \Delta_{k1} \\ \omega 1_{j,k} + \alpha \times a_{j2} \times \Delta_{k2} \\ \omega 2_{j,k} + \alpha \times a_{j3} \times \Delta_{k3} \\ . \\ . \\ . \\ \omega b - 1_{j,k} + \alpha \times a_{jb} \times \Delta_{kb} \end{bmatrix} \tag{1.6}$$

This method is implemented back to the start of the hidden layer, each of these errors is stored in another matrix and passed up to the predecessor layer, and the predecessor layer uses these stored variables for its calculation of the new weights. Note that the new weight of the neuron is only set when $\omega b_{j,k}$ is calculated from the previous inputs in the batch.

4 Task 2

4.1 Methodology

The structure of task 2 is implemented as the same structure that was described in section 2. The goal of task 2 is to implement a neural network that learns from a data file that is provided called "data1.csv". The data file consists of a set of previous moves in the order of 4 previous moves and then the players move that won the next round.

Input data representation

The input to the neural network is therefore referred to as these 4 previous moves, which can be anything from "RRRR" to "SSSS", these moves will have to be converted into some numeric value for the neural network to be able to convert it into an output, to do this a special method needs to be implemented to assign a unique value to each character. This uniqueness was implemented by one-hot encoding, one hot encoding means that when a set of values are presented or parameters are presented one assigns a bit to each parameter meaning the length of the one-hot bit is equal to the size of the possible parameters. To put it into an example: the previous moves can only be a Rock "R", a Paper "P" or a Scissor "S". There are 3 possible options, therefore the bit length is 3, in the form ["R","P","S"]. if one character is being used that character position will be a one and the rest will be a zero. If the input is R the bit will be [1,0,0] and if the input P will be [0,1,0] and the input S will be [0,0,1].

By using the above method the input will be a bit string of length 12. so when the previous moves played are "RPRS" the bit string will be [1,0,0,0,1,0,1,0,0,0,0,1]. A bias value of 1 will then be added to this bit string and this bit string will then be passed into the neural network.

Output data representation

The output will also be a one-hot encoding method, the output of the neural network, therefore, has to be of length 3 meaning the output layer must consist of 3 neurons, the neurons will be normalized and the neuron with the highest value will then be considered as the one-hot bit and this bit will then be converted back to a "R","P" or "S".

The activation function

(as the derivative of the activation function). Many possible activation functions can be used in a neural network some of these activation functions can be seen below in figures 4.1.1 - 4.1.3 these activation functions are non-linear but for the case of this practical, the tanh(x) function was implemented. From the first figure 4.1.1 and the last figure 4.1.3 of the activation functions, one can see that the output is always 0 or greater, this means a neuron is not activated if the sum of the inputs is not greater than 0. However it would be better if a neuron can be negatively activated and centred around zero, this is where the tanh() figure 4.1.2 activation function comes in handy the activation function can train the neural network that if a certain condition occurs to NOT do something meaning a condition can hurt the output, decreasing the probability of

a certain action to be taken. This function is also centred around 0 which means the function itself is not biased to a certain action.

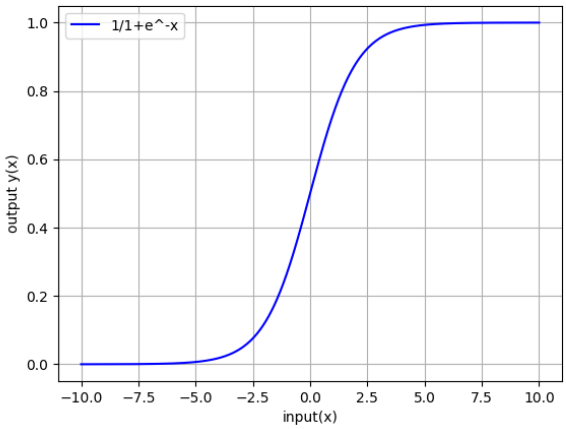

Figure 4.1.1: $\frac{1}{e^{-x}+1}$

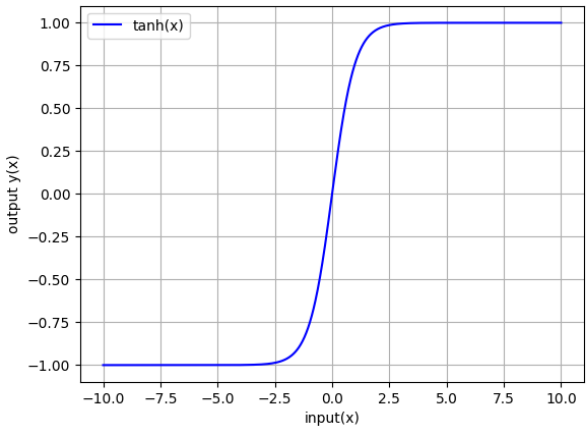

Figure 4.1.2: tanh(x)

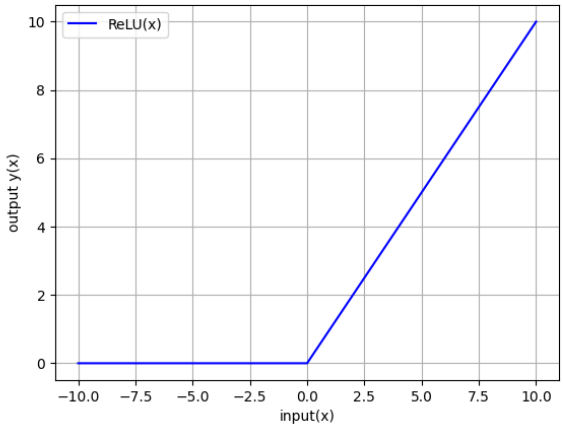

Figure 4.1.3: ReLU(x)

Batches or Epochs An Epoch is considered as an event, meaning one epoch is finished when one data input is converted into one data output. A batch is considered as a multiple of inputs converted into multiple outputs. There are advantages to each and disadvantages to each, the Epoch method is more sophisticated and accurate, however, it is slower and uses a bit more memory. A batch approach is less accurate, but it performs a bit faster and uses less memory. For the constructed neural network a batch and epoch can be passed into the neural network and the outputs calculated respectively. For learning, however, the batch method can produce sufficient results faster than the single Epoch method and with less memory space, it is important to notice that the batches can't be too big because then the inaccuracy factor is a disadvantage. the final decision on optimal batch sizes will be discussed in the discussion.

4.2 Results

In the results there are many simulations run the conditions for the learning process are as follows an x amount of hidden layers with a y amount of neurons in each layer, the neural network is initialized with random values as weights connecting the layers. The neural network's input can be presented as a batch input or as an epoch input. The neural network's backpropagation function trains the neural network at a set learning rate α. For figures:(4.2.1 $\longrightarrow$ 4.2.9) the dimensions of the neural network are 4 layers with 4 neurons each with a batch size of 10

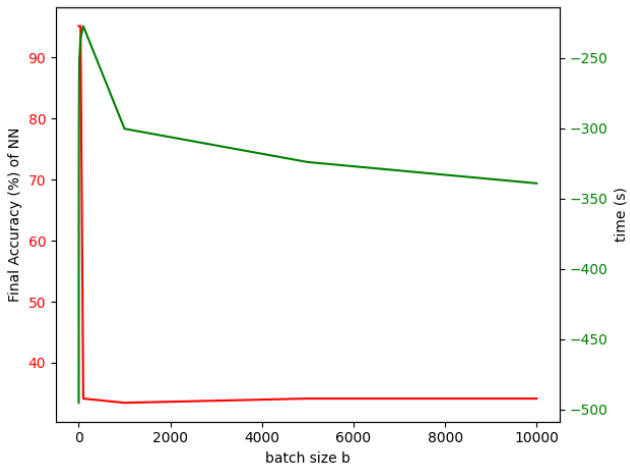

Figure 4.2.1: a figure of batch size with the batch's accuracy and time

Seeing in figure 4.2.1 that the bigger batch sizes cause a loss in final accuracy a smaller test will be done to see which possible batch sizes would satisfy the final accuracy requirement and use less time.

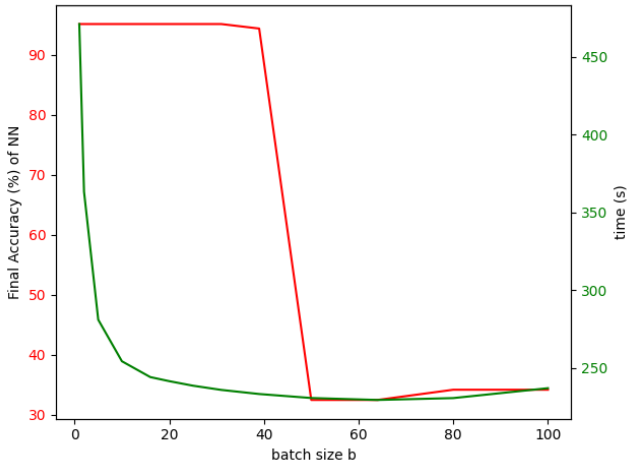

Figure 4.2.2: a figure of batch size with the batch's accuracy and time

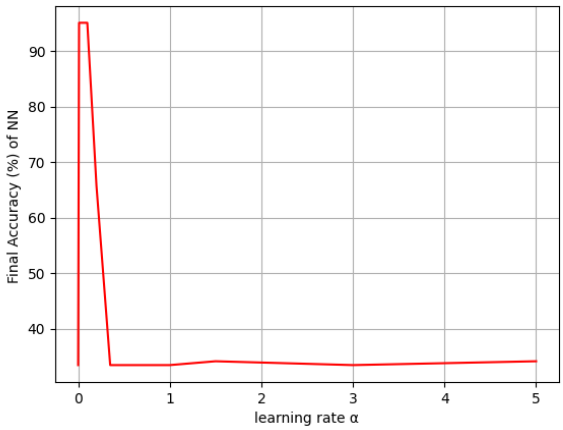

Figure 4.2.3: a figure of learning rates with the neural network's accuracy and time

noting any learning rate greater than 1 causes the neural network not to work at all. A finer test will be run on the error of each neuron at specific: "0,0.0005,0.05,1". For additional testing to confirm that the error decreases for a specific input, the input is manipulated to be the same input for each epoch: RPRS and the corresponding output is P.

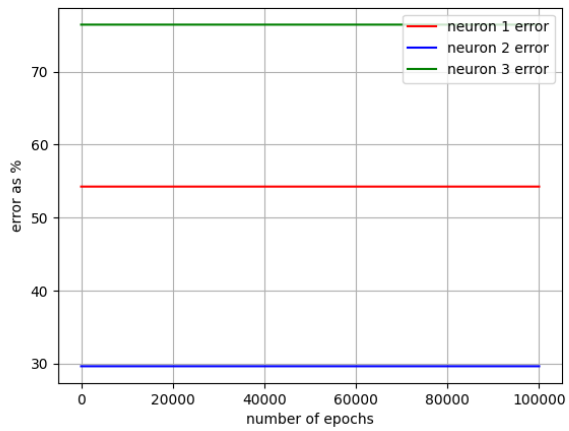

Figure 4.2.4: a figure of epochs and the corresponding error at a learning rate of 0

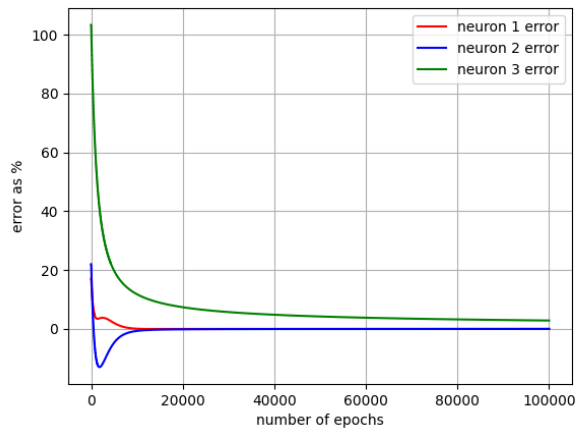

Figure 4.2.5: a figure of epochs and the corresponding error at a learning rate of 0.0005

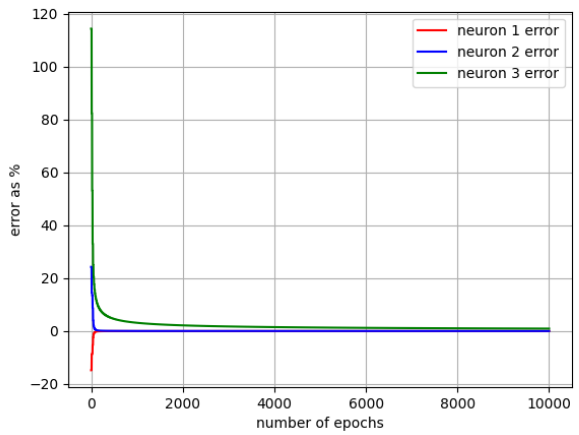

Figure 4.2.6: a figure of epochs and the corresponding error at a learning rate of 0.05

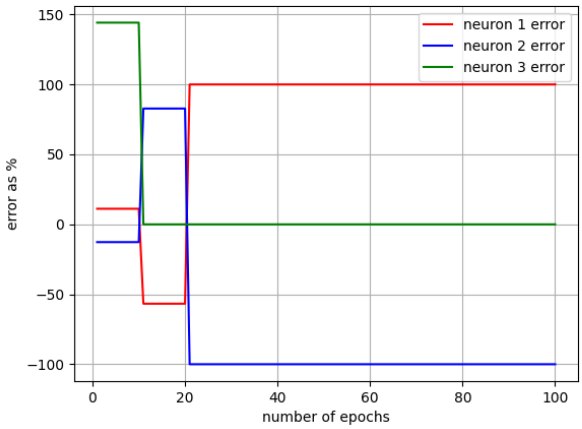

Figure 4.2.7: a figure of epochs and the corresponding error at a learning rate of 1

To confirm that the neural network work on the data file a simulation and calculation of error at a learning rate of 0.05 is run to confirm the neural network converges for the first elements in the data1.csv file, to further confirm that the neural network performs as expected to the data file was sorted and the error simulation was run for the first 100000 epochs of the neural network at different batch sizes.

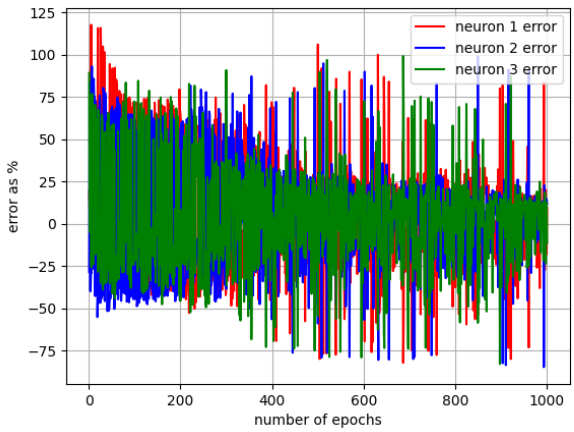

Figure 4.2.8: a figure of error as a percentage for the first 1000 iterations of the data1 file

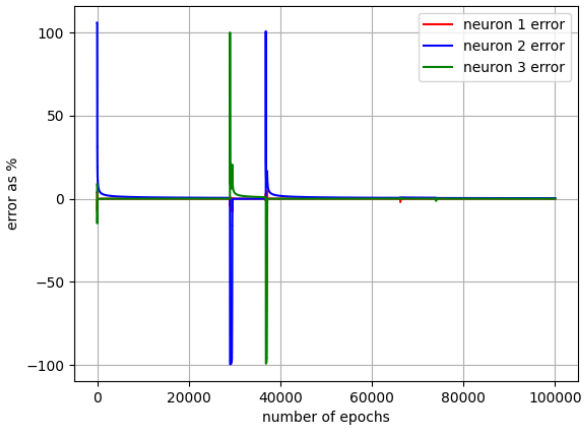

Figure 4.2.9: a figure of error as a percentage for the first 100000 iterations of the sorted data1 file at a learning rate of 0.05

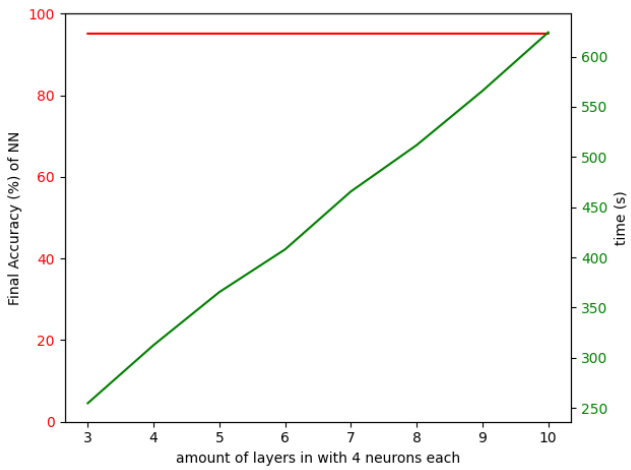

Figure 4.2.10: a figure of time and accuracy over layer sizes with 4 neurons each

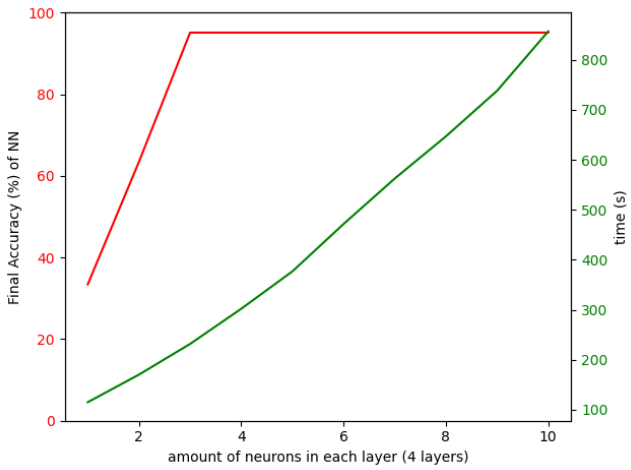

Figure 4.2.11: a figure of time and accuracy over neuron sizes with in 4 layers

For the implementation of Task2 to play like a bot the weights were generated using the training function at a learning rate of 0.05 and a batch size of 10. The structure of the neural

network is a 5x6 network, the reason for the learning rate, structure size and batch size will follow in the discussion, The batch size was changed for the bot to be compatible with the rps game structure. This bot is not spectacular to confirm that the training works and to prepare the bot for game implementation the learning rate will be 0, the bot will therefore not learn as it is playing but can be pre-trained for a certain game if required, The bot is pre-trained with the data1.csv file provided.The following results were obtained playing against :"only_rock","only_paper","only_scissors","beat_previous" at 100 matches against each with a match of size 10 000 rounds of the 400 matches the bot won 135 and won 170 rounds out of the 4 000 000.

4.3 Discussion

Batches

By using a batch it can immensely improve the time it takes for the neural network to learn, however it can come at a cost of accuracy as the batch sizes grow too large. On 4.2.1 the neural network is run at a set amount of batch sizes, one can see from this figure that the very large batches cost a lot in the final accuracy of the neural network, however, in figure 4.2.2 we can see that the trend at which the neural network performs at optimum accuracy stops at about a batch size of +-30, form this particular figure we can see how much the neural network spares on time and still obtains the optimal accuracy. The reason why the neural network can't run too many batch sizes is that for each iteration with a large batch size each value in the batch has been calculated using the initial weights, meaning each weight is changed according to the error, however each previous individual in the batch did not have an actual effect on the neural network, meaning it does not learn too often from the actual values, and the weights are changed after each input in the batch but each next input overshadows the previous input because the output of this input was not obtained using the changed weights and that error may harm the new weight generated by the previous output.

Learning rate

from 0.001 to 0.3, knowing that the optimal range is somewhere in between figures 4.2.4 to 4.2.7 were plotted from these we can see what happens to the error if it is tasked to train only one event and one can see what happens to the error if the learning rate is 0 and if the learning rate is 1, both the figures with learning rate 0.0005 and 0.05 performed good but to illustrate the lack in the speed of convergence the 0.0005 was chosen, one can see that it takes very long for the neural network to learn this one set of data where the learning rate of 0.05 converges much faster. Figures 4.2.8 and 4.2.9 were plotted to illustrate how the neural network learns on the data set provided for figure 4.2.8 the data is not sorted but one can see a light trend on convergence, the reason why convergence will not occur is that the data is scattered and it is not learning the same event in a row but scattered, however in figure4.2.9 the data file was sorted before training occurred and one can see that there is convergence as the inputs changes on each spike on the figure.

It is very important to choose a learning rate with the size of the neural network in the back of your mind, if the learning rate is too small and the neural network is very large the neural

network will take too long to converge to the correct weights and if the neural network is too small and the learning rate to big it will also cause the training to overshoot the estimated values.

Size of neural network

The size of the neural network plays a very big role in the use of the neural network increasing the size in the neural network can increase the time to learn immensely, it is also important to chose which item, neurons or layers to increase, both of them increase the time and number of weights in the neural network, but as can be seen in figure 4.2.10 the increase in layers causes time and weights to increase linearly, while the increase in neurons in each layer, see figure 4.2.11, increases in weights and time exponentially, it is more efficient to increase the amount of layers rather than increasing the neurons, increasing the size of the neural network will only pay off if the problem grows more complex, but for a rock paper and scissor game the decided dimensions are one input layer,3 hidden layers and 1 output layer with 6 neurons in the input and hidden layers, using this configuration the total amount of weights in the neural network are 204, which is enough should a bot be moderately complex and should a bot be not too complex.

a Unlearning neural network as a bot

The bot as it is implemented is quite useless unless it is pre-trained for a specific game, otherwise, it will keep on losing because the weights are not rewarded or punished if there is an error. Using a neural network without it being able to learn is not the optimal way to use it...see section 5 Discussion

5 Task 3

5.1 Methodology

For the implementation of task 3, the bot from task2 was used and the online learning capabilities were enabled, leaving the neural network with space to stretch its legs. For the implementation of the online neural network, the number of rounds that occur before learning from an input (n) can vary between 1 and 100 the neural network is also initialized with the same parameters of the neural network in task2, it was trained at a learning rate of 0.05 and has 5 layers with 6 neurons in each, it has been pre-trained with the data from the provided csv file.

5.2 Results

The exact same game as in section 4, Results, were run with the exact same dimensions,parameters and against the exacts same bots, however the difference this time was that learning was enabled. This time the bot won 400 out of the 400 matches and won 3 978 540 out of the 4 000 000 rounds.

The following table consist games against each of the bots that were provided at a n of 100.

Table 1: Table of games played against each agent with n = 100

bot played	matches won [%] by task3 bot	Rounds won [%] by task3 bot	Rounds lost [%]	time consumed [s] @16 threads
beat_common_prob	10	33	37.4	47.6
beat_common	100	53.1	23.2	47.5
beat_previous	100	74.4	5.8	46.92
breakable	64	34.7	33.9	46.76
never_repeat	66	35.3	33.7	45.71
only_paper	100	98.3	0	45.84
only_rock	100	99	0	48.31
only_scissors	100	96	0	46.83
random_play	48	33.3	33.3	45.69
really_smart_bot	0	4.5	90	116.05
sequence_random	63	35.7	34.3	46.95
task2 bot	100	53.8	1.4	97.83

The following table consist games against each of the bots that were provided at a n of 10.

Table 2: Table of games played against each agent with n = 10

bot played	matches won [%] by task3 bot	Rounds won [%] by task3 bot	Rounds lost [%]	time consumed [s] @ 16 threads
beat_common_prob	38	33.6	33.9	57.37
beat_common	100	50.3	49.2	54.67
beat_previous	100	71.2	14.1	57.05
breakable	94	35.9	32.5	58.10
never_repeat	98	36.5	32.2	54.21
only_paper	100	99.8	0	52.93
only_rock	100	99.9	0	54.97
only_scissors	100	99.6	0	55.36
random_play	49	33.3	33.3	55.09
really_smart_bot	0	15.5	66.8	122
sequence_random	94	63.2	32.5	52.86
task2 bot	100	49	48.9	95.47

The following table consist games against each of the bots that were provided at a n of 1.

Table 3: Table of games played against each agent with n = 1

bot played	matches won [%] by task3 bot	Rounds won [%] by task3 bot	Rounds lost [%]	time consumed [s] @ 16 threads
beat_common_prob	58	33.5	33.4	137.26
beat_common	100	68	26.8	133.83
beat_previous	100	98	1.3	138.96
breakable	100	48.5	25.8	131.28
never_repeat	100	46.6	26.7	134.66
only_paper	100	100	0	134.41
only_rock	100	100	0	131.57
only_scissors	100	100	0	134.08
random_play	60	33.5	33.3	134.09
really_smart_bot	0	12	64.3	175.95
sequence_random	100	44.8	27.9	132.16
task2 bot	100	97.6	0.4	175.57

5.3 Discussion

Games that were played

As can be seen in the first part of the Results we can see that once online learning is enabled the neural network results and the outcome of the games played changes dramatically. from tables 1, 2 and 3 we can see the results of games played against the learning neural network and other bots, using the latter of the tables we can see that the best possible results are found here, the neural network as a whole can be seen improving as learning is enabled, meaning the backpropagation function works as it is intended to. The neural network achieves overall good results when compared to other bots 11 out of 12 bots gets won consistently, probably because

they are not too complex when it comes to their game structure, however with the really smart bot, it is considered to be very smart, it constantly dominates against the neural network, this may be because it uses an approach that deliberately tries to confuse or miss-train the neural network. it consistently wins the neural network 100 matches to none, the reason why the neural network can't beat this bot is probably that this bot is too complex to solve in the configuration ran, making the configuration complex enough to solve this bot will be pointless because it will increase the time to compute immensely. The beat common prob bot also performs well against the neural network and may even sometimes win the neural network, the reason for this is because the beat common prob also uses a bit of learning technology and uses the probability of a move to be played as well. The random play also performs well but the neural network can still recognize a pattern, this is because it cannot be truly random.

sampling rate

Table 1 is run at a sampling rate of 100 meaning after each 100 epochs the system passes the epochs into the neural network for training, this means the neural network trains a bit less, for table 2 the neural network's sampling rate is set at 10 meaning after each 10 iteration the neural network learns, for table 3 the sampling rate is set at 1 meaning the neural network trains consistently after each epoch, the reason why the neural network obtains better results with a lower sampling rate is because each of the moves that will be played is affected by each previous move that is played where when n is greater than one each n set of moves are determined by the previous n set of moves that were played, using the n = 10 example, 10 moves are played on the same configuration and the learning occurs meaning if the neural network trained and it would need to change a few weights after iteration one of 10, these changes do not occur and the following 9 moves can be lost because those changes were not made immediately, fortunately the bot is efficient enough to run on a sampling rate of 1, the cost of making the sampling rate small, is time. It is important to know your opponent when choosing the sampling rate for example the results from only rock did not change much as the sampling rate decreased, however for beat common prob it changed the outcome from an overall loss to an overall victory with a smaller sampling rate.

learning rate, and Neural Network configuration

As explained in the creation and initial training of the neural network, it is very important to precisely choose the learning rate for the neural network to perform at its best, it is also important not to over-train or under-train the neural network for an implementation of a bot, if over-training is done and another game is played the trained values of the previous game will negatively affect the current game played and it may take to long to learn from the new game because the previous game is trained too much, under-training is when the neural network is not trained enough, meaning it does not have a set game structure, meaning it still plays a bit randomly, this can be to the neural network's advantage or disadvantage. It is very important to train the neural network just enough. It is also important to keep the structure of the neural network in mind, not making it too complex but just complex enough with the correct learning rate, one must note that the more weights there is, the higher the learning rate needs to be because the weights will be much more and learning will be a bit slower. Choosing a learning rate too large will cause the neural network to over-train and overshoot weight changes while making the learning rate too small will cause the neural network to learn too slowly.

6 Conclusion

From the results and discussions from section 4 and section 5 we can conclude that a neural network is indeed a suitable option for a bot when implementing it for a game type structure especially when there are patterns involved in the game, we can also conclude that the neural network performs as it is intended to and that it can consistently win bots that it plays against.

It can also be concluded that the size of the neural network is very important and that it carries a relationship with the complexity of the problem, it is also very important to choose a learning rate that would satisfy the speed of learning required with the specific setup. It can also be concluded that it is more time-efficient to increase the number of layers rather than increasing the number of neurons. Better training performance can also be achieved when implementing batches, there are batch sizes where optimal performance is achieved and the most time is saved when compared to one epoch at a time.

it can also be concluded that it is very important to train the neural network enough, not too much and not too little, the type of training can also have a spectacular effect on the performance of the neural network if the training method miss-trains the neural network the first few rounds of online playing will not go to the advantage of the neural network. The backpropagation function and activation functions are the heart of the neural network and without these, the neural network will not be able to perform the way it does. Brain damage was not tested with the neural network as it was not implemented to cope with changes like these but to use all weights and neurons to obtain optimal performance and efficiency. The neural network while training can recognize patterns in other bot's moves and compensate for that and win the match.

Bibliography

[1] B. I. C. Education, "What are neural networks?" [Online]. Available: https://www.ibm.com/cloud/learn/neural-networks#toc-what-are-n-2oQ5Vepe

[2] M. Ghosh, "Importance of activation functions in neural networks," Feb 2020. [Online]. Available: https://medium.com/swlh/importance-of-activation-functions-in-neural-networks-bc39964311dd#:~:text=Activationfunctionsareextremelyimportantforconstructinganeuralnetwork.&text=Onlytheneuronswithsome,non-linearityinthenetwork.